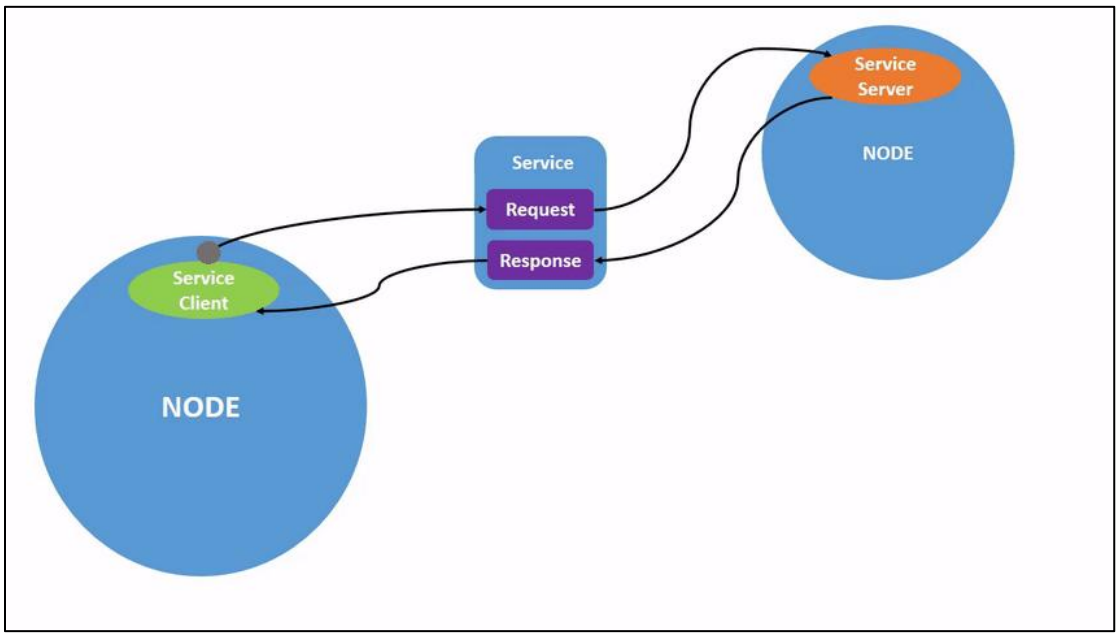
第 9 课 ROS2 服务说明

1.服务通讯简介

服务通讯是一种基于请求响应的通信模型，在通信双方中，客户端发送请求数据到服务端，服务端相应结果给客户端。



从服务的实现机制上来看，这种你问我答的形式叫做客户端/服务器模型，简称为 CS 模型，客户端在需要某些数据的时候，针对某个具体的服务，发送请求信息。服务器端收到请求之后，就会进行处理并反馈应答信息。

这种通信机制在生活中也很常见，比如我们经常浏览的各种网页，此时你的电脑浏览器就是客户端，通过域名或者各种操作，向网站服务器发送请求，服务器收到之后返回需要展现的页面数据。

2.创建接口

1) 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令，“**cd ros2_ws/src/demo_interfaces/**”指令，切换到 demo_interfaces 功能包中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/demo_interfaces/
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$
```

2） 输入“**mkdir srv**”指令，创建 srv 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ mkdir srv
```

3） 输入“**cd srv**”指令，进入 srv 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ cd srv
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/srv$
```

4） 输入指令“**gedit AddInts.srv**”复制下面程序粘贴到文本当汇总，“**ctrl+s**”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/srv$ gedit AddInts.srv
```

```
int32 num1

int32 num2

---

int32 sum
```



5） 先输入“**cd ..**”返回上一级文件夹

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/srv$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$
```

6） 再输入“**gedit CMakeLists.txt**”：

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ gedit CMakeLists.txt
```

复制下面程序粘贴到文本当中，“**ctrl+s**”保存并退出。

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces( ${PROJECT_NAME}
```

```
  "msg/String.msg"

  "srv/AddInts.srv"

)
```

```
28   # uncomment the line when this package is not in a git rep
29   #set(ament_cmake_cpplint_FOUND TRUE)
30   ament_lint_auto_find_test_dependencies()
31 endif()
32
33
34 find_package(rosidl_default_generators REQUIRED)
35 rosidl_generate_interfaces( ${PROJECT_NAME}
36   "msg/String.msg"
37   "srv/AddInts.srv"
38 )
39 ament_package()
```

3.创建服务

3.1 创建服务端

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“cd ros2_ws/src/”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入“**ros2 pkg create service_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**service_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create service_demo --build-type ament_python --dependencies rclpy
going to create a new package
package name: service_demo
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
```

3） 输入“**cd service_demo/service_demo/**”指令，切换到 service_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd service_demo/service_demo/
ubuntu@ubuntu:~/ros2_ws/src/service_demo/service_demo$
```

4） 输入指令“**gedit service_server.py**”编辑程序，复制下面程序粘贴到文本当

中，“**ctrl+s**”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/service_demo/service_demo$ gedit service_server.py
```

```
import rclpy

from rclpy.node import Node

from demo_interfaces.srv import AddInts



# 定义一个继承自 Node 的 MinimalService 类

class MinimalService(Node):



    # 类的初始化方法

    def __init__(self):

        # 调用 Node 类的初始化方法，设置节点名称为'minimal_service'

        super().__init__('minimal_service')



        # 创建一个服务，提供 AddInts 类型的服务，服务名称为'add_two_ints'，回调函数为 add_two_ints_callback

        self.srv = self.create_service(AddInts, 'add_two_ints', self.add_two_ints_callback)



    # 定义服务回调函数

    def add_two_ints_callback(self, request, response):

        # 在日志中记录接收到的请求的 num1 和 num2
```

```
        self.get_logger().info('Incoming request\nnum1: %d num2: %d' % (r
equest.num1, request.num2))



        # 计算并设置响应的 sum 字段

        response.sum = request.num1 + request.num2



        # 返回响应

        return response



# 主函数

def main():

    # 初始化 ROS 2 节点

    rclpy.init()



    # 创建 MinimalService 对象

    minimal_service = MinimalService()



    # 进入 ROS 2 节点的事件循环

    rclpy.spin(minimal_service)



    # 关闭 ROS 2 节点

    rclpy.shutdown()
```

```
# 如果该脚本是主程序，则执行 main 函数
if __name__ == '__main__':
    main()
```

5） 接着输入指令“**chmod +x service_server.py**”，并按下回车，为保存的 service_server.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/service_demo/service_demo$ chmod +x service_server.py
```

3.2 创建发布者

1） 输入指令“**gedit service_client.py**”编辑程序，复制下面程序粘贴到文本当中，“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/service_demo/service_demo$ gedit service_client.py
```

```
import sys
import rclpy
from rclpy.node import Node
from rclpy.logging import get_logger
from demo_interfaces.srv import AddInts

# 定义一个继承自 Node 的 MinimalClient 类
class MinimalClient(Node):

    # 类的初始化方法
    def __init__(self):
```

```
    # 调用 Node 类的初始化方法，设置节点名称为'minimal_client'
    super().__init__('minimal_client')

    # 创建一个服务客户端，连接到名为'add_two_ints'的 AddInts 服务
    self.cli = self.create_client(AddInts, 'add_two_ints')

    # 等待服务连接，最多等待 1 秒
    while not self.cli.wait_for_service(timeout_sec=1.0):
        self.get_logger().info('service is connecting...')

# 发送服务请求的方法
def send_request(self):
    # 创建一个 AddInts 请求对象
    request = AddInts.Request()

    # 从命令行参数中获取两个整数，并设置到请求对象中
    request.num1 = int(sys.argv[1])
    request.num2 = int(sys.argv[2])

    # 异步调用服务，并获取一个 Future 对象
    self.future = self.cli.call_async(request)
```

```
# 主函数

def main():

    # 检查命令行参数是否包含两个整数

    if len(sys.argv) != 3:

        get_logger("rclpy").error("请提供两个整数值")

        return



    # 初始化 ROS 2 节点

    rclpy.init()



    # 创建 MinimalClient 对象

    minimal_client = MinimalClient()



    # 发送服务请求

    minimal_client.send_request()



    # 阻塞等待服务调用完成

    rclpy.spin_until_future_complete(minimal_client, minimal_client.futu
re)



    try:

        # 获取服务调用的响应
```

```
        response = minimal_client.future.result()



        # 打印响应结果

        minimal_client.get_logger().info("请求结果：sum = %d" % response.s
um)

    except Exception:

        # 打印请求失败的错误信息

        minimal_client.get_logger().error("请求失败")



    # 销毁节点对象

    minimal_client.destroy_node()



    # 关闭 ROS 2 节点

    rclpy.shutdown()



# 如果该脚本是主程序，则执行 main 函数

if __name__ == '__main__':

    main()
```

2） 输入指令“**chmod +x service_client.py**”，并按下回车，为保存的 service_client.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/service_demo/service_demo$ chmod +x service_client.py
```

4.setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 service_client.py 和 service_server.py 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/service_demo/service_demo$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/service_demo$
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。

```
ubuntu@raspberrypi:~/hiwonder_ws/src/service_demo$ vim setup.py
```

复制下面程序粘贴到文本当中，“ctrl+s”保存并退出。

```
'service_server = service_demo.service_server:main',

'service_client = service_demo.service_client:main'
```

```
18      description='TODO: Package description',
19      license='TODO: License declaration',
20      tests_require=['pytest'],
21      entry_points={
22          'console_scripts': [
23          'service_server = service_demo.service_server:main',
24          'service_client = service_demo.service_client:main'
25          ],
26      },
27  )
```

5.编译运行

1） 新建一个命令行终端，然后输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 run service_demo service_server**”，并按下回车，启动 service_server 服务端。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run service_demo service_server
```

5） 然后再新建一个命令行终端，输入指令，“**cd ros2_ws**”,进入到工作空间当中。

```
ubuntu@ubuntu:~$ cd ros2_ws/
```

6） 接着输入指令“**source install/setup.bash**”

```
ubuntu@ubuntu:~/ros2_ws$ source install/setup.bash
ubuntu@ubuntu:~/ros2_ws$
```

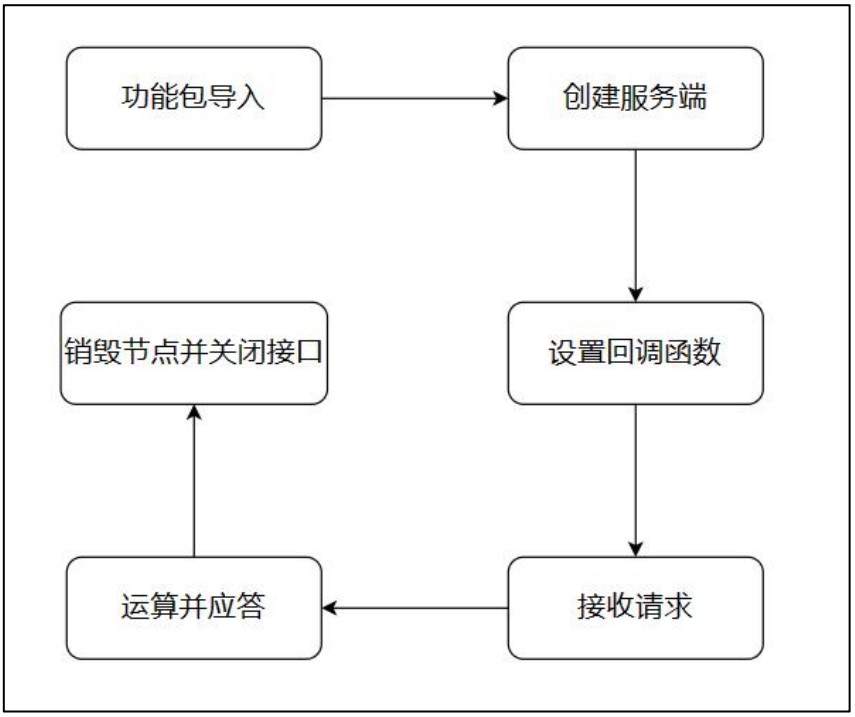
7） 输入指令“**ros2 run service_demo service_client 1 2**”，并按下回车，启动 **service_client** 客户端发送数字 1 和数字 2 的运算请求，服务端收到数字 1 和数字 2 的运算请求后，将运算后的结果发送给客户端。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run service_demo service_client 1 2
[INFO] [1709029712.287087826] [minimal_client]: 请求结果：sum = 3
```

6.程序分析

6.1 服务端

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个名为 minimal_service 的服务端，该服务端将收到的请求进行运算，把运算后的结果进行应答响应。在程序运行时，首先会初始化 ROS2 节点，然后创建 MinimalService 对象并进入 ROS 2 节点的事件循环。当程序被中断时，会销毁节点对象并关闭 ROS 2 节点。

◆ 主函数

```
# 主函数
def main():
    # 初始化ROS 2节点
    rclpy.init()

    # 创建MinimalService对象
    minimal_service = MinimalService()

    # 进入ROS 2节点的事件循环
    rclpy.spin(minimal_service)

    # 关闭ROS 2节点
    rclpy.shutdown()
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalService()，然后在 ROS2 节点的事件循环执行 minimal_service。

◆ MinimalService 类

```python
# 定义一个继承自Node的MinimalService类
class MinimalService(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法，设置节点名称为'minimal_service'
        super().__init__('minimal_service')

        # 创建一个服务，提供AddInts类型的服务，服务名称为'add_two_ints'，回调函数为add_two_ints_callback
        self.srv = self.create_service(AddInts, 'add_two_ints', self.add_two_ints_callback)

    # 定义服务回调函数
    def add_two_ints_callback(self, request, response):
        # 在日志中记录接收到的请求的num1和num2
        self.get_logger().info('Incoming request\nnum1: %d num2: %d' % (request.num1, request.num2))

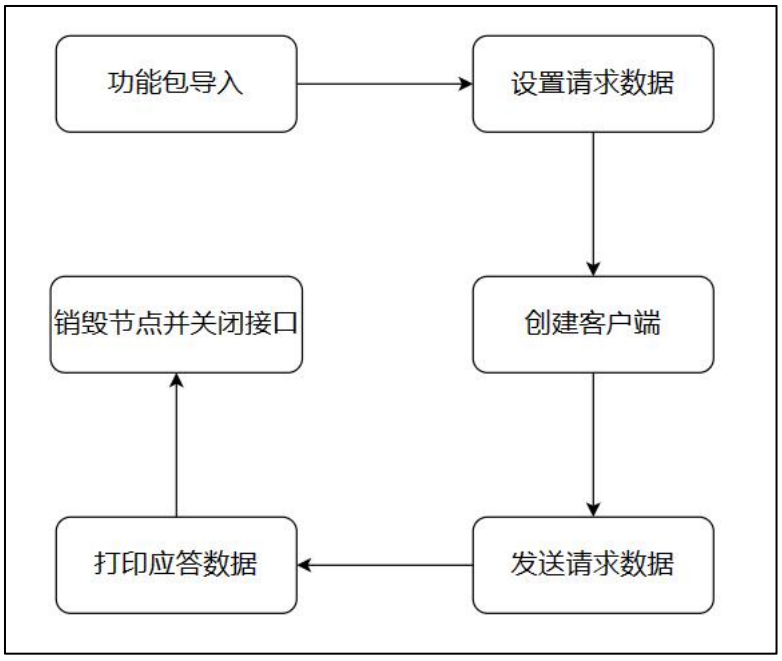
        # 计算并设置响应的sum字段
        response.sum = request.num1 + request.num2

        # 返回响应
        return response
```

首先创建了一个名为 minimal_service 的节点，然后创建一个服务，提供 AddInts 类型的服务，服务名称为'add_two_ints'，add_two_ints_callback()回调函数里面接收请求的两个数据，并对其进行运算处理，把运算后的结果响应返回。

6.2 客户端

根据实现的效果，梳理程序的过程逻辑，如下图所示：



创建了一个名为 minimal_client 的客户端，该客户端连接到连接到名为'add_two_ints'的 AddInts 服务，将请求的两个数据发送出去。在程序运行时，首先会初始化 ROS2 节点，然后创建 MinimalClient 对象，发送服务请求，并将响应的结果打印

到出来。当程序被中断时，会销毁节点对象并关闭 ROS 2 节点。

◆ 主函数

```
def main():
    # 检查命令行参数是否包含两个整数
    if len(sys.argv) != 3:
        get_logger("rclpy").error("请提供两个整数值")
        return

    # 初始化ROS 2节点
    rclpy.init()

    # 创建MinimalClient对象
    minimal_client = MinimalClient()

    # 发送服务请求
    minimal_client.send_request()

    # 阻塞等待服务调用完成
    rclpy.spin_until_future_complete(minimal_client, minimal_client.future)

    try:
        # 获取服务调用的响应
        response = minimal_client.future.result()

        # 打印响应结果
        minimal_client.get_logger().info("请求结果：sum = %d" % response.sum)
    except Exception:
        # 打印请求失败的错误信息
        minimal_client.get_logger().error("请求失败")

    # 销毁节点对象
    minimal_client.destroy_node()

    # 关闭ROS 2节点
    rclpy.shutdown()
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 MinimalClient()，接着发送服务请求，最后处理响应的数据。

◆ MinimalClient 类

```python
# 定义一个继承自Node的MinimalClient类
class MinimalClient(Node):

    # 类的初始化方法
    def __init__(self):
        # 调用Node类的初始化方法，设置节点名称为'minimal_client'
        super().__init__('minimal_client')

        # 创建一个服务客户端，连接到名为'add_two_ints'的AddInts服务
        self.cli = self.create_client(AddInts, 'add_two_ints')

        # 等待服务连接，最多等待1秒
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('service is connecting...')

    # 发送服务请求的方法
    def send_request(self):
        # 创建一个AddInts请求对象
        request = AddInts.Request()

        # 从命令行参数中获取两个整数，并设置到请求对象中
        request.num1 = int(sys.argv[1])
        request.num2 = int(sys.argv[2])

        # 异步调用服务，并获取一个Future对象
        self.future = self.cli.call_async(request)
```

首先创建了一个名为 minimal_client 的节点，然后 创建一个服务客户端，连接到名为'add_two_ints'的 AddInts 服务，然后在 send_request()函数，将命令行终端的两个参数作为请求对象的数据，发送请求。